Experiment 5 : Random Forest for Car Safety Predic

Aim: Predict the safety of a car from its features using a Random Forest classifier.

In [1]:
```
# A52 Dipanshu Ambilkar
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OrdinalEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
```

In [2]:
```
X, y = fetch_openml(name="car", version=1, return_X_y=True, as_frame=True
print("Shape:", X.shape, "| safety classes:", sorted(y.unique()))
```

```
Shape: (1728, 6) | safety classes: ['acc', 'good', 'unacc', 'vgood']
```

```
/home/hatch/workspace/practicals-nov1/venv/lib/python3.12/site-packages/sklea
datasets/_openml.py:1061: UserWarning: Version 1 of dataset car is inactive,
meaning that issues have been found in the dataset. Try using a newer version
this URL: https://openml.org/data/v1/download/21/car.arff
  warn(
```

In [3]:
```
print(X.head())
```

```
  buying  maint doors persons lug_boot safety
0  vhigh  vhigh     2       2    small    low
1  vhigh  vhigh     2       2    small    med
2  vhigh  vhigh     2       2    small   high
3  vhigh  vhigh     2       2      med    low
4  vhigh  vhigh     2       2      med    med
```

In [4]:
```
print("Cars per safety class:")
print(y.value_counts())
```

```
Cars per safety class:
class
unacc    1210
acc       384
good       69
vgood      65
Name: count, dtype: int64
```

In [5]:
```
X = OrdinalEncoder().fit_transform(X)
print("Encoded shape:", X.shape)
```

```
Encoded shape: (1728, 6)
```

In [6]:
```
print("First row after encoding:", X[0])
```

```
First row after encoding: [3. 3. 0. 0. 2. 1.]
```

In [7]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=7)
print("Train size:", len(X_train), "| Test size:", len(X_test))
```

```
Train size: 1382 | Test size: 346
```

In [8]:
```
model = RandomForestClassifier(n_estimators=100, random_state=7)
print("Model created:", model)
```

```
Model created: RandomForestClassifier(random_state=7)
```

In [9]:
```
model.fit(X_train, y_train)
print("Random Forest trained with 100 trees")
```

```
Random Forest trained with 100 trees
```

In [10]:
```
pred = model.predict(X_test)
print("First 10 predictions:", list(pred[:10]))
```

```
First 10 predictions: ['unacc', 'unacc', 'unacc', 'unacc', 'unacc', 'acc', 'u
'good', 'unacc', 'unacc']
```

In [11]:
```
print("Accuracy:", round(accuracy_score(y_test, pred), 4))
```

```
Accuracy: 0.9884
```

In [12]:
```
print("Classification report:")
print(classification_report(y_test, pred))
```

```
Classification report:
              precision    recall  f1-score   support

         acc       0.99      0.97      0.98        77
        good       1.00      0.75      0.86         8
       unacc       0.99      1.00      1.00       246
       vgood       0.94      1.00      0.97        15

    accuracy                           0.99       346
   macro avg       0.98      0.93      0.95       346
weighted avg       0.99      0.99      0.99       346
```